

Linux Class 8 - Practice Task

Practising the grep options and the awk column commands

Submitted by: Shivakumar Muradi

Date: 30 September 2026

This is the practice I did after class 8. I made a folder called practice8.

The two files I made

grep and awk need two different kinds of file, so I made two.

The first one is notes.txt. It is just 8 plain sentences about different things, some about cricket and some about cities and some about the weather. I put the word good in it in three different ways on purpose: as good, as GOOD and as Good, and once inside the word goodbye. That way I can see what the grep options really do.

The second one is class8.txt, which has 5 employee rows with an id, a name, a department and a salary. I used Indian names. I also left one blank line in the middle of it, because the notes have a command for skipping blank lines and I wanted to see that work.

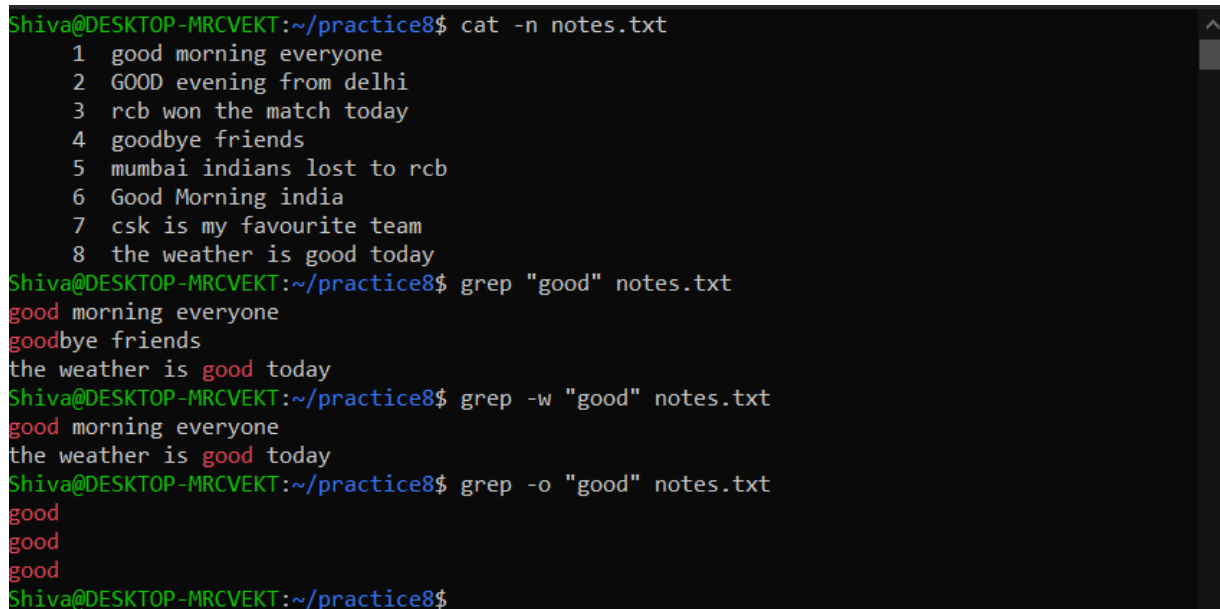
grep - finding the pattern

grep searches inside a closed file and prints the lines where the pattern is. I ran cat -n notes.txt first so the line numbers are there to check against.

```
grep "good" notes.txt
```

```
grep -w "good" notes.txt
```

```
grep -o "good" notes.txt
```



```
Shiva@DESKTOP-MRCVEKT:~/practice8$ cat -n notes.txt
 1 good morning everyone
 2 GOOD evening from delhi
 3 rcb won the match today
 4 goodbye friends
 5 mumbai indians lost to rcb
 6 Good Morning india
 7 csk is my favourite team
 8 the weather is good today
Shiva@DESKTOP-MRCVEKT:~/practice8$ grep "good" notes.txt
good morning everyone
goodbye friends
the weather is good today
Shiva@DESKTOP-MRCVEKT:~/practice8$ grep -w "good" notes.txt
good morning everyone
the weather is good today
Shiva@DESKTOP-MRCVEKT:~/practice8$ grep -o "good" notes.txt
good
good
good
Shiva@DESKTOP-MRCVEKT:~/practice8$
```

Screenshot 1: the file, then plain grep, then -w and -o.

Plain grep gave 3 lines, and one of them was goodbye friends. That surprised me, because I was looking for the word good and goodbye is a different word. But grep is only looking at letters, so good inside goodbye still counts.

-w fixed that. It means whole word, and it gave only 2 lines. goodbye was left out this time.

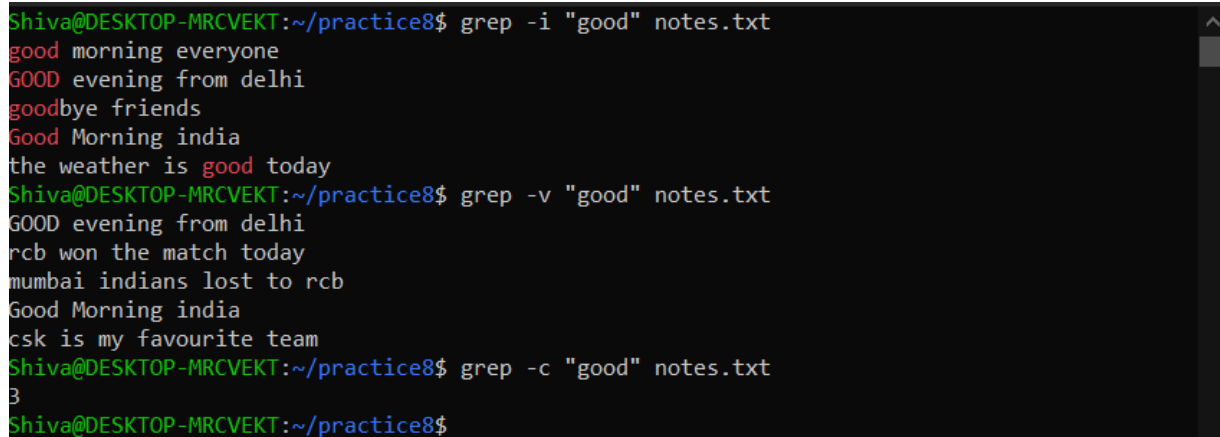
-o printed only the word itself, three times, one under the other, instead of the full lines. So -o is for when I want to see the matches and not the sentences.

Ignoring case, turning it around, and counting

```
grep -i "good" notes.txt
```

```
grep -v "good" notes.txt
```

```
grep -c "good" notes.txt
```



```
Shiva@DESKTOP-MRCVEKT:~/practice8$ grep -i "good" notes.txt
good morning everyone
GOOD evening from delhi
goodbye friends
Good Morning india
the weather is good today
Shiva@DESKTOP-MRCVEKT:~/practice8$ grep -v "good" notes.txt
GOOD evening from delhi
rcb won the match today
mumbai indians lost to rcb
Good Morning india
csk is my favourite team
Shiva@DESKTOP-MRCVEKT:~/practice8$ grep -c "good" notes.txt
3
Shiva@DESKTOP-MRCVEKT:~/practice8$
```

Screenshot 2: -i finds 5 lines, -v shows the other 5, and -c just gives the number.

-i ignores capital letters, so now GOOD evening from delhi and Good Morning india came too. That is 5 lines instead of 3.

-v is the opposite of normal grep. It printed the lines that do not have good in them. It is still case sensitive, so the GOOD and Good lines are in this list, which looked wrong for a second until I remembered that -v and -i are two separate things.

-c did not print any line at all, only the number 3. So -c counts lines, not words. Even if a line had the word twice it would still count as one.

Line numbers, a two word pattern, and many patterns

```
grep -n "good" notes.txt
```

```
grep -n -i "good" notes.txt
```

```
grep "good morning" notes.txt
```

```
grep -e "morning" -e "rcb" -e "good" notes.txt
```

```

Shiva@DESKTOP-MRCVEKT:~/practice8$ grep -n "good" notes.txt
1:good morning everyone
4:goodbye friends
8:the weather is good today
Shiva@DESKTOP-MRCVEKT:~/practice8$ grep -n -i "good" notes.txt
1:good morning everyone
2:GOOD evening from delhi
4:goodbye friends
5:Good Morning india
8:the weather is good today
Shiva@DESKTOP-MRCVEKT:~/practice8$ grep "good morning" notes.txt
good morning everyone
Shiva@DESKTOP-MRCVEKT:~/practice8$ grep -e "morning" -e "rcb" -e "good" notes.txt
good morning everyone
rcb won the match today
goodbye friends
mumbai indians lost to rcb
the weather is good today
Shiva@DESKTOP-MRCVEKT:~/practice8$

```

Screenshot 3: line numbers, then two options together, then a two word pattern, then three patterns at once.

-n puts the line number and a colon in front, so I got 1:, 4: and 8:. Putting -n and -i together works fine and gave 5 numbered lines.

For a pattern with a space in it, like good morning, the quotes are what matter. Without quotes the shell would think morning is the file name. It found only line 1.

-e is how to look for more than one thing. Each pattern needs its own -e. I asked for morning or rcb or good and it gave me 5 lines, in the order they appear in the file and not in the order I typed the patterns.

Option	What it does	On my file, searching for good
(none)	Lines that have the pattern anywhere	3 lines, goodbye counted too
-w	Only when it is a whole word	2 lines, goodbye left out
-o	Prints the word, not the line	good printed 3 times
-i	Ignores capital letters	5 lines, GOOD and Good included
-v	The lines that do NOT have it	5 lines, the opposite set
-c	Counts the lines instead	just the number 3
-n	Puts the line number in front	1: 4: 8:
-e	Lets me give more than one pattern	5 lines for morning, rcb or good

awk - picking out columns

awk looks at a file as rows and columns. The columns are just the words separated by spaces, and awk calls them \$1, \$2 and so on. In my class8.txt, \$1 is the id, \$2 is the name, \$3 is the department and \$4 is the salary.

```

awk '{print}' class8.txt
awk '{print $2}' class8.txt
awk '{print $2, $4}' class8.txt

```

```

Shiva@DESKTOP-MRCVEKT:~/practice8$ cat -n class8.txt
 1 101 Arjun IT 50000
 2 102 Priya HR 45000
 3 103 Rahul IT 60000
 4
 5 104 Vikram Finance 55000
 6 105 Sneha HR 48000
Shiva@DESKTOP-MRCVEKT:~/practice8$ awk '{print}' class8.txt
101 Arjun IT 50000
102 Priya HR 45000
103 Rahul IT 60000

104 Vikram Finance 55000
105 Sneha HR 48000
Shiva@DESKTOP-MRCVEKT:~/practice8$ awk '{print $2}' class8.txt
Arjun
Priya
Rahul

Vikram
Sneha
Shiva@DESKTOP-MRCVEKT:~/practice8$ awk '{print $2, $4}' class8.txt
Arjun 50000
Priya 45000
Rahul 60000

Vikram 55000
Sneha 48000
Shiva@DESKTOP-MRCVEKT:~/practice8$

```

Screenshot 4: the whole file, then only the names, then the names with the salaries.

awk '{print}' with nothing inside just gives the whole file, like cat. Then \$2 gave me the five names, and \$2, \$4 gave name and salary side by side.

The comma matters. It puts a space between the two columns. Without the comma awk sticks them together with nothing in between, so I would get Arjun50000.

NF - how many columns are there

NF means number of fields, which is awk's word for columns. It is not a column itself, it is a count, so it does not take a dollar sign.

```
awk '{print NF, $4}' class8.txt
```

```
awk '{print NF, $0}' class8.txt
```

```
awk '{print NF}' class8.txt
```

```

Shiva@DESKTOP-MRCVEKT:~/practice8$ awk '{print NF, $4}' class8.txt
4 50000
4 45000
4 60000
0
4 55000
4 48000
Shiva@DESKTOP-MRCVEKT:~/practice8$ awk '{print NF, $0}' class8.txt
4 101 Arjun IT 50000
4 102 Priya HR 45000
4 103 Rahul IT 60000
0
4 104 Vikram Finance 55000
4 105 Sneha HR 48000
Shiva@DESKTOP-MRCVEKT:~/practice8$ awk '{print NF}' class8.txt
4
4
4
0
4
4
Shiva@DESKTOP-MRCVEKT:~/practice8$

```

Screenshot 5: the count with the salary, the count with the whole line, and the count on its own.

Every real row gave 4, because every row has four columns. The blank line gave 0, which makes sense because a blank line has no words in it at all.

\$0 is a useful one. It means the whole line, all the columns together. So NF, \$0 is the count followed by the original line, like numbering the file by how many words each line has.

NR - which row am I on

NR means number of records, which here just means the line number. It also takes no dollar sign.

```

awk '{print NR, $2}' class8.txt
awk '{print NR, $0}' class8.txt
awk 'END{print NR}' class8.txt

```

```

Shiva@DESKTOP-MRCVEKT:~/practice8$ awk '{print NR, $2}' class8.txt
1 Arjun
2 Priya
3 Rahul
4
5 Vikram
6 Sneha
Shiva@DESKTOP-MRCVEKT:~/practice8$ awk '{print NR, $0}' class8.txt
1 101 Arjun IT 50000
2 102 Priya HR 45000
3 103 Rahul IT 60000
4
5 104 Vikram Finance 55000
6 105 Sneha HR 48000
Shiva@DESKTOP-MRCVEKT:~/practice8$ awk 'END{print NR}' class8.txt
6
Shiva@DESKTOP-MRCVEKT:~/practice8$

```

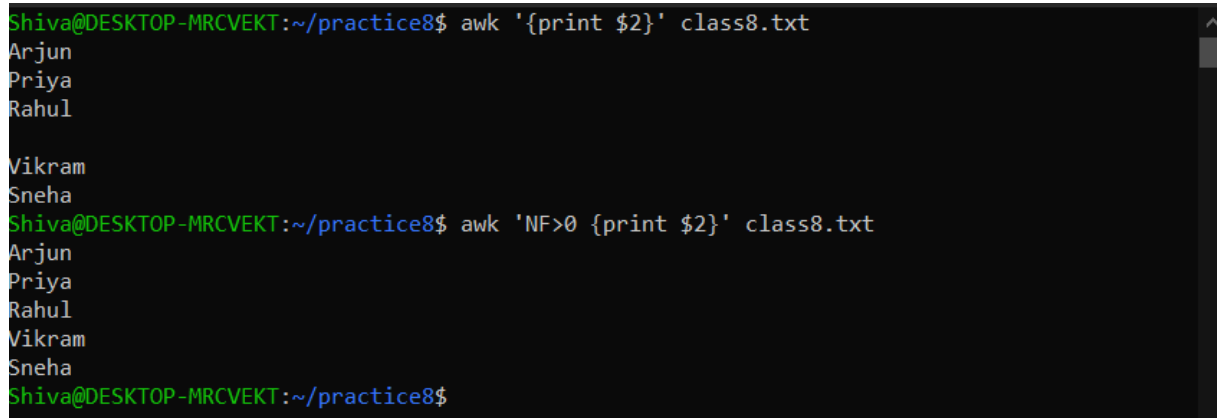
Screenshot 6: line number with the name, line number with the whole line, and the total at the end.

The first two worked like cat -n. The third one is different. END means do this once, after the last line has been read. By then NR has stopped counting and holds the final number, so it printed 6. My file has 5 employees but 6 lines, because of the blank line I left in.

Skipping the blank line

This is the last command in the notes and it finally cleans up my blank line.

```
awk '{print $2}' class8.txt  
awk 'NF>0 {print $2}' class8.txt
```



```
Shiva@DESKTOP-MRCVEKT:~/practice8$ awk '{print $2}' class8.txt  
Arjun  
Priya  
Rahul  
  
Vikram  
Sneha  
Shiva@DESKTOP-MRCVEKT:~/practice8$ awk 'NF>0 {print $2}' class8.txt  
Arjun  
Priya  
Rahul  
Vikram  
Sneha  
Shiva@DESKTOP-MRCVEKT:~/practice8$
```

Screenshot 7: with and without the blank line in the middle.

The first one printed a gap where the blank line is, because awk still reads that line and \$2 on it is nothing. The second one has NF>0 written before the flower brackets. That is a condition, and it means only do the printing when the line has at least one column. Blank lines have 0 columns, so they are skipped and the five names come out together.

So anything written before the brackets is a test, and the brackets are what to do when the test is true. That is the part of awk I did not understand before this practice.

Command	What it printed
awk '{print}' class8.txt	The whole file, same as cat.
awk '{print \$2}' class8.txt	Only the names, which is column 2.
awk '{print \$2, \$4}' class8.txt	Name and salary together.
awk '{print NF, \$4}' class8.txt	How many columns the line has, then the salary.
awk '{print NF, \$0}' class8.txt	The count in front of the whole line.
awk '{print NF}' class8.txt	Just the counts. The blank line gives 0.
awk '{print NR, \$2}' class8.txt	Line number and the name.
awk '{print NR, \$0}' class8.txt	Line number and the whole line.
awk 'END{print NR}' class8.txt	6, the total number of lines.
awk 'NF>0 {print \$2}' class8.txt	The names, with the blank line skipped.